

Projet Pédagogique : Mini-Framework TypeScript Réactif

Objectif

Créer un **mini-framework TypeScript** côté client permettant de développer une application Web réactive, en utilisant l'ensemble des Design Patterns vus en cours. Le framework doit être construit **sans aucune bibliothèque externe** (hormis Vite.js pour le bundling) et doit aller jusqu'à l'implémentation des **Observables**.

Consignes générales

Technologies autorisées

Technologie	Statut
TypeScript	✓ Obligatoire
Vite.js	✓ Uniquement pour le bundling
Node.js	✓ Pour l'environnement de développement
Docker + Docker Compose	✓ Obligatoire
HTML / CSS	✓ Autorisé
DOM API (vanilla)	✓ Autorisé — c'est le cœur du framework
Autres librairies (React, Vue, Svelte, jQuery, Lodash, etc.)	✗ Interdites

Infrastructure

- **Docker Compose** obligatoire pour servir l'application (serveur HTTP statique type Nginx).
- L'application doit fonctionner dans un navigateur en `localhost`.
- Le `Dockerfile` et le `docker-compose.yml` doivent être présents à la racine du projet.

Architecture du projet

```
projet/
├── src/
│   ├── core/           # Cœur du framework (patterns)
│   │   ├── factory.ts
│   │   ├── builder.ts
│   │   ├── singleton.ts
│   │   ├── strategy.ts
│   │   └── observer.ts
│   ├── components/     # Composants réutilisables
│   ├── store/          # Gestion d'état
│   ├── router/         # Routage
│   ├── http/           # Client HTTP
│   ├── utils/          # Utilitaires
│   ├── app.ts          # Point d'entrée de l'application
│   └── main.ts         # Point d'entrée Vite
├── public/
│   └── index.html
├── tests/
│   └── *.test.ts
└── Dockerfile
```

```
├─ docker-compose.yml
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
└─ README.md
```

💡 Exemples d'implémentation par Design Pattern

Chaque pattern doit résoudre un problème **concret côté client**. Voici des exemples détaillés de ce qui est attendu pour chacun.

🏗️ Infrastructure et application réactive 1 pt

- Projet Vite + TypeScript configuré avec `strict: true`
- Fichier `Dockerfile` et `docker-compose.yml` présents et fonctionnels
- L'application affiche un texte dans le navigateur via `localhost`
- Utilisation de l'API DOM standard (`document.createElement` , `document.body.appendChild` , etc.)
- Création interactive d'éléments HTML (boutons, divs, etc.) via le framework
- Gestion d'événements (clics, saisies) avec `addEventListener`
- Application de styles CSS dynamiques
- Mise à jour du DOM en réponse aux actions utilisateur

Exemple : L'utilisateur clique sur un bouton → le texte change, un style s'applique.

🏗️ Builder — Construction d'éléments DOM 1 pt

Problème : Construire des éléments DOM complexes avec du texte, des classes, des styles, des événements et des enfants sans constructeurs surchargés.

Structure attendue :

```
class TagBuilder {
  constructor(private tag: string) {}

  // Méthodes with* : ajoutent une propriété et retournent this (fluent interface)
  withText(text: string): this { /* à implémenter */ return this; }
  withClass(className: string): this { /* à implémenter */ return this; }
  withStyle(property: string, value: string): this { /* à implémenter */ return this; }
  withEvent(event: string, handler: EventListener): this { /* à implémenter */ return this; }
}

withChild(child: HTMLElement): this { /* à implémenter */ return this; }

// Méthodes without* : retirent une propriété
withoutClass(className: string): this { /* à implémenter */ return this; }
withoutEvent(event: string): this { /* à implémenter */ return this; }

// build : construit et retourne l'élément HTMLElement final
```

```
build(): HTMLElement { /* à implémenter */ }  
}
```

Utilisation concrète :

```
// Carte de produit construite avec le Builder  
const card = new TagBuilder('div')  
  .withClass('card')  
  .withChild(  
    new TagBuilder('img')  
      .withClass('card-image')  
      .withStyle('width', '200px')  
      .build()  
  )  
  .withChild(  
    new TagBuilder('div')  
      .withClass('card-body')  
      .withText('Description du produit')  
      .build()  
  )  
  .withChild(  
    new TagBuilder('button')  
      .withClass('btn-primary')  
      .withText('Acheter')  
      .withEvent('click', () => alert('Acheté !'))  
      .build()  
  )  
  .build();
```

Factory — Création rapide d'éléments HTML 1 pt

Problème : Le Builder est puissant mais verbeux pour des cas simples. Créer un bouton, un `<hr>` ou une `<img>` ne devrait pas requérir une chaîne de méthodes. La Factory offre une API concise pour instancier rapidement les éléments HTML les plus courants.

Pistes de réflexion :

- Définir une **interface** `Tag` représentant un élément HTML avec un nom de balise, des attributs et des événements, ainsi qu'une méthode `toHtml()` retournant l'élément DOM créé.
- Créer des **classes concrètes** (`ButtonTag`, `DivTag`, `ImageTag`, `HorizontalRuleTag`, `InputTag`, `HeadingTag`, `SpanTag`, `ParagraphTag` ...) qui implémentent cette interface. Chaque classe encapsule la logique de création de **son** élément DOM.
- Concevoir une **TagFactory** qui, given un type et une configuration, retourne l'instance `Tag` correspondante — sans `if / switch`, en utilisant un `Record<ElementType, Tag>`.
- Réfléchir à comment les attributs (`class`, `id`, `src`, `href` ...) et événements (`click`, `input`, `submit` ...) sont passés à chaque tag.
- Chaque tag peut accepter un contenu spécifique : un `text` pour les boutons/paragraphes, un `src` pour les images, un tableau de `children` pour les conteneurs, etc.

Idée clé : La Factory et le Builder se complètent — la Factory pour la création rapide d'éléments simples, le Builder pour la composition complexe d'éléments imbriqués.

Singleton — Gestion d'état et configuration 1 pt

Problème : Partager un état ou une configuration unique à travers toute l'application sans le passer manuellement à chaque composant.

Structure attendue :

```
// Configuration globale de l'application
class AppConfig {
  private static instance: AppConfig; // instance unique
  private config: Record<string, string> = {};

  private constructor() {} // constructeur privé

  static getInstance(): AppConfig {
    // à implémenter : retourne l'instance unique, la crée si elle n'existe pas
  }

  set(key: string, value: string): void { /* à implémenter */ }
  get(key: string): string | undefined { /* à implémenter */ }
}

// Store global d'état applicatif
class AppStore {
  private static instance: AppStore;
  private state: Record<string, unknown> = {};

  private constructor() {} // TODO : accepter une StorageStrategy en
  paramètre (voir Strategy)

  static getInstance(): AppStore { /* à implémenter */ }
  getState<T>(key: string): T | undefined { /* à implémenter */ }
  setState(key: string, value: unknown): void { /* à implémenter */ }
  // TODO : setStrategy(strategy: StorageStrategy) – permet de changer le backend de
  stockage
}
```

Utilisation concrète :

```
AppConfig.getInstance().set('apiUrl', '/api');
const store = AppStore.getInstance();
store.setState('theme', 'dark');
store.setState('user', { name: 'Alice', role: 'admin' });
```

Strategy — Gestion du stockage (Volatile, LocalStorage, IndexedDB) 5 pts

Problème : L'AppStore (Singleton) a besoin de persister ses données, mais le mode de stockage peut varier selon le contexte (mémoire volatile pour le dev, localStorage pour la simplicité, IndexedDB pour les gros volumes). Il faut pouvoir changer de stratégie à l'exécution sans modifier le Store.

Pistes de réflexion :

- Définir une **interface StorageStrategy** commune décrivant les opérations de stockage (get, set, remove, clear ...).

- Implémenter **3 stratégies concrètes** :
 - `VolatileStorage` — stocke en mémoire (Map ou objet), les données disparaissent au rechargement
 - `LocalStorageAdapter` — persiste dans le `localStorage` du navigateur
 - `IndexedDBStorage` — persiste dans une base `IndexedDB`, adaptée aux gros volumes
- L' **AppStore** (Singleton) accepte une `StorageStrategy` et lui délègue la persistance. Il expose une méthode `setStrategy()` pour changer de backend à l'exécution.
- Se poser la question : comment gérer le caractère **asynchrone** d'`IndexedDB` vs le caractère **synchrone** de `localStorage` et du Map en mémoire ? (Piste : uniformiser via des Promesses)

Idee clé : Le Singleton (`AppStore`) gère l'état global, la Strategy détermine **où** et **comment** cet état est persisté. Les deux patterns travaillent ensemble.

Observer — Réactivité et événements 2 pts

Problème : Notifier plusieurs parties de l'application lorsqu'une valeur change, sans couplage fort entre l'émetteur et les récepteurs.

Structure attendue :

```
// Observable générique (type T = type de la valeur transportée)
class Observable<T> {
  private subscribers: Array<(value: T) => void> = [];

  // S'abonner aux notifications, retourne une fonction de désabonnement
  subscribe(callback: (value: T) => void): () => void {
    // à implémenter : ajouter callback à la liste, retourner unsubscribe
  }

  // Émettre une nouvelle valeur à tous les abonnés
  next(value: T): void {
    // à implémenter : notifier chaque abonné avec la valeur
  }
}

// Exemples d'observables concrets
const counter = new Observable<number>(0);
const theme = new Observable<'light' | 'dark'>('light');
const todos = new Observable<Array<{ id: number; text: string; done: boolean }>>([]);
```

Utilisation concrète :

```
// Mettre à jour le DOM automatiquement quand la valeur change
const unsubscribe = counter.subscribe(value => {
  document.getElementById('counter')!.textContent = String(value);
});

theme.subscribe(currentTheme => {
  document.body.className = currentTheme;
});

counter.next(1); // → le DOM se met à jour
theme.next('dark'); // → le body change de classe
```

```
// Se désabonner quand on n'a plus besoin des notifications
unsubscribe();
```

Mise en pratique des patterns

Les sections suivantes demandent d'**utiliser** les patterns implémentés précédemment pour construire les fonctionnalités du framework.

Système de composants avec cycle de vie 1 pt

- Création d'un système de **composants** avec cycle de vie
- Méthodes de cycle de vie : `onMount` , `onUpdate` , `onDestroy`
- Un composant encapsule son état, son rendu et son comportement
- Possibilité de créer des composants réutilisables

Architecture :

```
class Component {
  onMount(): void;
  onUpdate(): void;
  onDestroy(): void;
  render(): HTMLElement;
}
```

Réactivité Observable → DOM 1 pt

- Liaison d'un **Observable** à un élément du DOM
- Lorsque l'Observable émet une nouvelle valeur, le DOM se met à jour automatiquement
- Implémentation sans Virtual DOM (mise à jour directe et ciblée)
- Support de la désinscription pour éviter les fuites mémoire

Exemple :

```
const count = new Observable<number>(0);
count.subscribe(value => { /* mise à jour du DOM */ });
count.next(1); // → le DOM se met à jour
```

Routage client (History API / Navigation API) 1 pt

- Implémentation d'un **Router** utilisant l'**History API** (`history.pushState` , `history.replaceState` , `popstate`) ou, si le navigateur le supporte, la **Navigation API** (`navigation.navigate` , `navigation.currentEntry`)
- Utilisation d'**Observer** pour écouter les changements de route

- Affichage conditionnel de pages/composants selon la route active
 - Gestion des paramètres de route (optionnel)
-

Store global (état centralisé) 1 pt

- Implémentation d'un **Store** global (Singleton) combiné avec **Observer**
 - L'état est stocké dans une instance unique
 - Les composants s'abonnent aux changements d'état
 - Mécanisme d'actions pour modifier l'état de manière prévisible
 - Typage fort de l'état
-

Gestion de formulaires avec validation 1 pt

- Binding bidirectionnel : les inputs modifient l'état, et vice-versa
 - Validation de formulaires avec **Strategy** (plusieurs stratégies : required, email, minLength, pattern)
 - Affichage des messages d'erreur
 - Soumission du formulaire avec récupération des données
-

Client HTTP 1 pt

- Implémentation d'un **Client HTTP** basé sur `fetch`
 - Support des méthodes GET, POST, PUT, DELETE
 - Gestion des erreurs et des timeouts
 - Interceptors (optionnel, via Strategy)
-

Composition de composants (slots) 1 pt

- Support de **composants enfants** (slots / children)
- Communication parent-enfant (props descendantes, événements remontants)
- Un composant peut être réutilisé dans différents contextes
- Exemple : créer un `Card`, `List`, `Modal` avec des enfants personnalisables

Architecture :

```
render() {  
  return this.builder  
    .withChild(new Header({ title: 'Mon app' }))  
    .withChild(this.children) // slot  
    .withChild(new Footer())  
    .build();  
}
```

 Application de démonstration complète 1 pt

- Application CRUD complète utilisant **tous les patterns** du framework
 - Exemples : gestionnaire de tâches, prise de notes, catalogue de produits
 - Au moins 3 pages / vues différentes (accueil, liste, détail)
 - Navigation via le routeur
 - Gestion d'état via le Store
 - Communication HTTP avec une API (ou mock)
 - Validation de formulaires
 - Interface utilisateur soignée (CSS)
-

 Tests unitaires et qualité du code 1 pt

- Tests unitaires pour **chaque pattern** (Factory, Builder, Singleton, Strategy, Observer)
- Tests pour le système de composants, le Store, le Router
- Tests pour les directives et la réactivité
- Utilisation de Vitest (pas de Jest)
- Au moins un test par méthode publique
- Mock du DOM si nécessaire (`jsdom`)
- **JSDoc** sur toutes les classes, interfaces et méthodes publiques
- Code propre : pas de `any` , pas de code mort, pas de commentaire superflu
- Respect des conventions TypeScript strict
- Architecture cohérente et bien organisée
- Déploiement Docker fonctionnel (vérifié)